Deep reinforcement learning for the dynamic vehicle dispatching problem: An event-based approach

Edyvalberty Alenquer Cordeiro*

Department of Applied Mathematics and Statistics, Federal University of Ceará, Fortaleza, Brazil

Anselmo R. Pitombeira-Neto†

Department of Industrial Engineering, Federal University of Ceará, Fortaleza, Brazil

July 17, 2023

Abstract

The dynamic vehicle dispatching problem corresponds to deciding which vehicles to assign to requests that arise stochastically over time and space. It emerges in diverse areas, such as in the assignment of trucks to loads to be transported; in emergency systems; and in ride-hailing services. In this paper, we model the problem as a semi-Markov decision process, which allows us to treat time as continuous. In this setting, decision epochs coincide with discrete events whose time intervals are random. We argue that an event-based approach substantially reduces the combinatorial complexity of the decision space and overcomes other limitations of discrete-time models often proposed in the literature. In order to test our approach, we develop a new discrete-event simulator and use double deep q-learning to train our decision agents. Numerical experiments are carried out in realistic scenarios using data from New York City. We compare the policies obtained through our approach with heuristic policies often used in practice. Results show that our policies exhibit better average waiting times, cancellation rates and total service times, with reduction in average waiting times of up to 50% relative to the other tested heuristic policies.

*Email: edyalenquer@gmail.com

†Email: anselmo.pitombeira@ufc.br

1 Introduction

Taxi services play a crucial role in urban areas, which currently have the highest population densities [1]. Residents of these areas have increasingly favored transportation options over purchasing their vehicles [2]. The transformation of urban mobility through the use of apps such as Uber, DiDi, and Lyft has positive impacts related to sustainability and generation of income in the provision of services of this nature [3, 4].

To ensure efficient service and meet demands promptly, the system must maintain continuous availability of vehicles to respond to the incoming ride requests. Additionally, minimizing waiting time is essential to prevent customer cancellations, as customers have tolerance thresholds for how long they are willing to wait. The optimization of service levels involves addressing three different problems: vehicle dispatching, vehicle relocation, and price setting [2].

In this paper, we address the dynamic vehicle dispatching problem, which corresponds to assigning vehicles to requests that arise stochastically over time and space. Previous works have treated this problem as a Markov decision process (MDP) due to its dynamic and stochastic nature. Given the MDP formulation of the problem, the goal is to approximate an optimal decision policy evaluated by a specified objective function (or loss function).

Most previous studies assume that decision epochs occur at fixed time intervals, during which the system collects information on vehicles of the fleet and pending ride requests. This information forms the state at that particular time. The decision involves determining the optimal assignment of available vehicles to waiting requests. This can be alternatively formulated as a problem of "matching" vehicles and requests. Although this matching-based MDP formulation has been popular in the literature, it exhibits several limitations:

1. At every decision epoch, the decision space corresponds to all possible matchings of vehicles to requests. The optimal decision is ideally obtained through solving an NP-hard combinatorial optimization problem, formulated as an integer nonlinear program. It is often reduced to a linear assignment problem due to its intractability, which can be solved efficiently but simplifies reality. (For example, the relaxed model ignores the waiting tolerance threshold of the customers, which is an important constraint in the real applications.)

2. It is difficult to account for the possibility of both the driver and the customer rejecting the proposed ride in the mathematical programming model for the matching problem. For this reason, this matching-based approach often assumes that all proposed rides will be accepted.

3. Since decision epochs occur at fixed time intervals, one must decide on the length of the time interval, i.e., it is a parameter that has to be previously specified or tuned by computational experimentation. Short time intervals may waste system resources while long time intervals may result in wasting valuable time that could have been used to make good assignments earlier, thus reducing customers' waiting time.

In this paper, we seek to overcome the aforementioned limitations of the matching-based approach to the dynamic vehicle dispatching problem. In particular, our main contributions are the following:

1. We propose an event-based approach for the dynamic vehicle dispatching problem that can reduce the idle time of the system and take into account some of the characteristics of the real problem that are usually disregarded in other works. More importantly, since we do not have to evaluate the set of all feasible matchings at each decision epoch, our approach can considerably minimize the complexity of the decision space.

2. We develop a new simulation environment that uses real data to sample customers' demand and simulate the dynamics of the environment.

3. We develop two deep reinforcement learning agents that are our decision-makers, each being responsible for a specific event.

4. We show that this new approach for the dynamic vehicle dispatching problem is promising by comparing the result of a large number of experiments with heuristic policies often used in practice.

In contrast to most previous studies, which use a discrete-time approach based on the MDP framework, we develop a continuous-time approach, which is more appropriate for this problem as requests may arrive to the system and the vehicles can finish services at random times. To account for continuous time, we model the problem as a semi-Markov decision process. In this model, decision epochs are not fixed at specific time intervals but occur at random points in time. These decision epochs correspond to two types of events: when a new request arrives in the system; or when a vehicle completes a service. This approach enables us to make decisions as soon as possible without having to specify a fixed length for the time interval between decision epochs. Additionally, since decision epochs only occur when necessary, the overall utilization of system resources is reduced.

Due to the large scale of real instances of the problem, obtaining optimal policies using exact methods is computationally infeasible, since these methods rely on enumerating all the states in the state space. To obtain good policies,

we employ deep reinforcement learning and train two agents to make decisions corresponding to the two abovementioned decision events using the double deep q-learning algorithm. While our primary objective is to minimize customers' average waiting times, we also aim to avoid an increase in the cancellation rate or a decrease in drivers' income. Therefore, we simultaneously monitor these performance measures when comparing different policies.

The structure of this paper is the following: In Section 2, we comment on related work on the dynamic vehicle dispatching problem; Section 3 revises fundamental theory on which our work is based; in Section 4 we present our formulation of the dynamic vehicle dispatching problem as a semi-Markov decision process; in Section 5 we detail our solution approach, which is based on reinforcement learning and simulation; Section 6 presents numerical results with the use of data from New York City; in Section 7 we discuss the results; finally, in Section 8 we draw some conclusions and suggest possible future works.

2 Related work

The dynamic vehicle dispatching problem (DVDP) is related with the class of dynamic fleet management and assignment problems in transportation. One of the earliest works to address a dynamic assignment problem in the context of transportation is due to Powell [5], who developed deterministic and stochastic mathematical programming models which address demand forecasts and repositioning of trucks. Approximate dynamic programming approaches to large-scale dynamic fleet management have been developed in the works of Simão et al. [6] and Godfrey and Powell [7]. In their idea, the classical linear assignment problem is extended to a multi period environment as a discrete-time Markov decision process. At each decision epoch, many vehicles are available for assignment to many tasks (loads, in the case of trucks), resulting in a large combinatorial decision space. To cope with this, one of the strategies is to use a linear approximation to the value function, which allows one to efficiently solve the decision problem as a linear program.

In the first years of the use of digital platforms for ride hailing, companies adopted simple policies for the dispatch of vehicles to customers requests. Liao [8] and Lee et al. [9] report on the adoption of simple myopic policies for the assignment of vehicles based on the shortest path time to arrive at a customer's location given current traffic conditions. The main disadvantage of these simple rules is that they act locally and myopically, not accounting for the global and future impact of decisions on system state, and therefore not optimizing for global performance measures.

Seow et al. [10] formulate the DVDP at each decision epoch as a linear

assignment problem with the objective of minimizing the sum of estimated current travel times from available vehicles to pending requests. Instead of solving the problem in a centralized way, the authors propose a collaborative algorithm in which groups of agents representing the drivers would negotiate and arrive at desirable assignments. The authors show through computational experiments that such approach outperforms simple rules applied by centralized systems.

Zhang et al. [11] formulate an assignment problem whose objective function is the sum of acceptance probabilities by drivers. Probabilities are estimated from a logistic regression model. As the objective function of the assignment problem is nonlinear, the authors use a heuristic to solve it. Bertsimas et al. [12] consider the ride-hailing problem as an online version of the dial-a-ride problem. They develop a re-optimization approach in which a static mixed-integer programming model is solved at fixed time intervals. The model includes all pending requests, idle vehicles and estimated travel times at the time of optimization and implement only the actions that can be applied before the next optimization epoch.

More recently, the DVDP has been addressed by using reinforcement learning (RL) techniques. Xu et al. [13] use an MDP model in which the service provider has to match requests and available vehicles at constant time intervals. To tackle the large state space, the authors assume that the global value function is separable in local value functions associated with the drivers, and learn a value function for a single driver, whose state space is much smaller. They use past data on taxi trips to learn the value function of a base policy used during data sampling via a tabular form of dynamic programming. Decisions at real time are taken by the service provider by solving a linear assignment problem whose cost function is the sum of the learned value functions of the vehicles.

In order to overcome limitations of the tabular representation, Wang et al. [14] develop an approach based on deep RL in which a deep q-network receives a pair (state, action) as input and outputs the q-value associated with the pair. The network is trained by using gradient descent with data from historical trips. At runtime, the learned q-values are used to compose the objective-function of a linear assignment problem to match vehicles and requests. A similar approach is developed in the work by Liang et al. [15]. Tang et al. [16] extend the MDP model to an SMDP and develop a new neural network architecture called "cerebellar value networks" to learn the value function. Most of these developments are summarized in Qin et al. [17].

Other works consider the task of vehicle repositioning, either separately or integrated with order dispatching. Miao et al. [18] develop a receding horizon control approach to repositioning of vehicles in anticipation of demand so as to balance the supply and demand and minimizing total idle cruising distance

of vehicles. Holler et al. [19] consider that decision epochs occur every time a vehicle becomes free, and then an action correspond to choose a pending request within a broadcast radius of the vehicle or reposition the vehicle in case no request is available within this radius. They train both DQNs and proximal policy optimization to learn policies. In their results, learned policies performed comparatively better relative to simple myopic policies such as assigning the request with shortest pickup distance or with the highest revenue.

Kullman et al. [20] have developed a deep RL approach to the DVDP with electric vehicles. At each decision epoch, a central provider must decide on which jobs to assign to each electric vehicle, such as pickup a pending request or reposition to a charging station. The authors use DQNs to learn the value function and compare the learned policies with a myopic reoptimization approach.

Liu et al. [21] developed a context-aware DQN-based approach to vehicle repositioning. In their work, the decision is to reposition current idle taxis to adjacent cluster locations given predicted demand in order to balance supply and demand. The matching of vehicles and requests is done by simply assigning the nearest idle vehicle. Tang et al. [22] developed an integrated approach to dispatching and repositioning. They use a neural network to represent the state-value function and use both offline and online training. A policy corresponds to solving a linear assignment problem whose objective function is given by the sum of temporal difference errors associated with a pair (vehicle, request).

In addition to the works directly related to the DVDP summarized in this section, in the next section we revise general concepts on which we base our approach.

3 Preliminaries

In this section, we revise the main theoretical concepts and ideas that underpin our work: Markov and semi-Markov decision processes, reinforcement learning and discrete-event simulation.

3.1 Markov and semi-Markov decision processes

Markov decision processes (MDPs) formalize situations in which an agent (or decision maker) makes a sequence of decisions over time under uncertainty. The sequence of possible states that an agent may observe constitutes a Markov stochastic process. MDPs have been studied in connection with stochastic dynamic programming and are often used as models to sequential decision problems [23, 24].

An MDP is defined by a tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$, in which $\mathcal{S}$ is a state set, $\mathcal{A}$ is an action set, P is a probability measure, $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is a reward function,

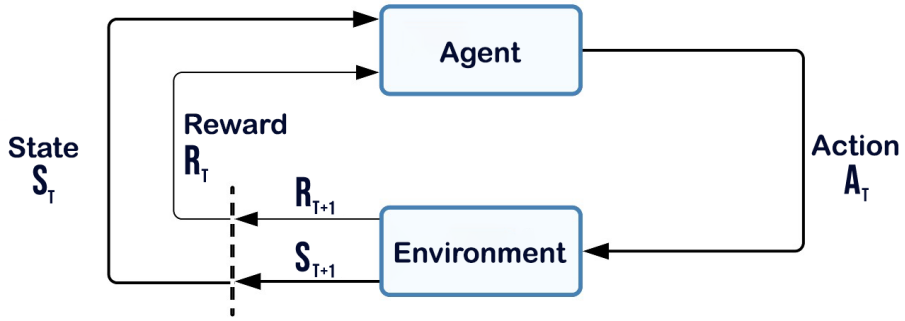

Figure 1: Representation of the dynamic interaction between MDP elements [25].

and $0 < \gamma \leq 1$ is a discount factor. The state set often represents the possible states of an environment and the action set is related to an agent (a decision maker) which interacts with the environment. At a given decision epoch, the environment is in a current state s. The agent chooses an action a, receives a reward $r = r(s, a)$ and the environment makes a transition to a new state s' according to a conditional probability distribution $p(s'|s, a)$. These components work together to represent the dynamics of the process, as illustrated in Figure 1.

A solution to an MDP is called a policy, which can be regarded as a sequence of decision rules. Policies can be deterministic or stochastic. When a policy is deterministic, it always takes the same action when a given state occurs. On the other hand, when a policy is stochastic, the action taken depends on a probability distribution over all possible actions. We are interested in deterministic stationary policies, which may be defined as functions from the state set $\mathcal{S}$ to the set of actions $\mathcal{A}$.

A Markov decision problem consists in finding a policy which optimizes an objective-function. In this paper, we work with infinite-horizon MDPs, whose objective-function is

$$v^\star(s) = \max_{\pi \in \Pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi(s_t)) \Big| s_0 = s\right], \quad \forall s \in \mathcal{S}, \tag{1}$$

s_0 is an initial state, $\pi : \mathcal{S} \to \mathcal{A}$ is a policy and Π is the class of deterministic stationary policies. The function $v^\star : \mathcal{S} \to \mathbb{R}$ is known as the optimal value function. Thus, an optimal policy $\pi^\star$ maximizes the objective-function (1) for all initial states.

The optimal value function $v^\star$ can be obtained by solving the *Bellman equation* [26]:

$$v^\star(s) = \max_{a \in \mathcal{A}_s} \{r(s,a) + \gamma \mathbb{E}[v^\star(s')]\}, \quad \forall s \in \mathcal{S},$$

in which $\mathcal{A}_s \subseteq \mathcal{A}$ is the set of feasible actions at state s. It can be shown that knowledge of $v^\star$ leads directly to an optimal policy by simply acting greedily in relation to the the optimal value function:

$$\pi^\star(s) \in \arg\max_{a \in \mathcal{A}_s} \{r(s,a) + \gamma \mathbb{E}[v^\star(s')]\}, \quad \forall s \in \mathcal{S}.$$

There are three main exact methods to obtain an optimal policy: value iteration [26], policy iteration [27] and linear programming [28, 29]. These methods have been applied to a variety of problems, such as robotics control [30], and game playing [31]. Value Iteration uses the Bellman equation to compute the optimal value function iteratively. Policy iteration combines policy evaluation and policy improvement steps, where the Bellman equation is used to evaluate the policy.

Semi-Markov decision processes (SMDPs) generalize MDPs by allowing decision epochs to occur at deterministic irregular time intervals or random time intervals [32–34]. SMDPs extend the class of problems which can be modeled by the MDP formalism as the following:

1. Decision epochs do not need to occur at every state transition and may be started by events corresponding to so called *decision states*.

2. Rewards may be a function of the time interval length.

3. The environment may remain at a given state for a variable time interval.

SMDPs also allow the decoupling of the *natural process* and the *decision process*. The natural process corresponds to the sequence of all states of the environment over time, while the decision process corresponds to the sequence of only the decision states. In the MDP formalism these two processes coincide.

Since we assume continuous time in SMDPs, the objective function must be modified as follows:

$$v^\star(s) = \max_{\pi \in \Pi} \mathbb{E}\left[\sum_{k=0}^{\infty} \int_{t_k}^{t_{k+1}} e^{-\beta t} r(s_k, \pi(s_k)) dt \middle| s_0 = s\right], \quad \forall s \in \mathcal{S}, \tag{2}$$

in which s_k is the state at decision epoch t_k. Notice in (2) that the time interval between consecutive decision epochs t_k and t_{k+1} is a random variable, so that we must integrate the reward function over the interval and $e^{-\beta t}$ is the limiting form of the discrete-time discount factor γ for continuous time, with $e^{-\beta} = \gamma$.

Let $R(s,a)$ be the expected reward between two consecutive decision epochs:

$$R(s,a) = \mathbb{E}\Big[\int_0^{\tau} e^{-\beta t} r(s,a)dt\Big]\Big\},$$

in which the expectation is taken relative to the probability distribution of soujourn times τ between decision epochs. Notice that, in contrast to MDPs, in SMDPs the rewards depend on the random time intervals between decision epochs.

The optimally Bellman equation for an SMDP can be written as

$$v^{\star}(s) = \max_{a \in \mathcal{A}_s} \Big\{R(s,a) + \mathbb{E}\Big[\int_0^{\infty} e^{-\beta t} v^{\star}(s')dt\Big]\Big\}, \quad \forall s \in \mathcal{S},$$

and the expectation is taken relative to the distribution of next states s'.

3.2 Reinforcement learning

Reinforcement learning (RL) has emerged as a collection of solution methods within the artificial intelligence community, specifically designed to address problems concerning the training of artificial agents in decision-making over time. RL theory encompasses the employment of Markov Decision Processes (MDP) and Semi-Markov Decision Processes (SMDP) as general mathematical models for solving RL problems. In recent years, RL has garnered considerable attention and acclaim due to its remarkable achievements in domains such as board games [35], robotics [36, 37], video games [38], and operations research [39–41]. RL algorithms exhibit several similarities to approximate dynamic programming techniques [42, 43] in various aspects.

RL is particularly valuable when dealing with sequential decision problems where the underlying model is unknown. To address such scenarios, RL employs approximation techniques. In many real-world applications, such as robotics or game playing, it is challenging, if not impossible, to model the environment and the consequences of actions precisely. In these cases, RL provides a framework for learning a policy or a value function that can guide the decision-making process based on experience gathered through trial-and-error interactions with the environment. RL makes use of Q-functions to overcome the need for a model:

$$q^{\star}(s,a) = R(s,a) + \mathbb{E}\Big[\int_0^{\infty} e^{-\beta t} v^{\star}(s')dt\Big], \quad \forall s \in \mathcal{S}, \forall a \in \mathcal{A}.$$

Q-functions satisfy a form of Bellman equation:

$$q^{\star}(s,a) = R(s,a) + \mathbb{E}\Big[\int_0^{\infty} e^{-\beta t} \max_{a' \in \mathcal{A}_{s'}} q^{\star}(s',a')dt\Big], \quad \forall s \in \mathcal{S}, \forall a \in \mathcal{A}.$$

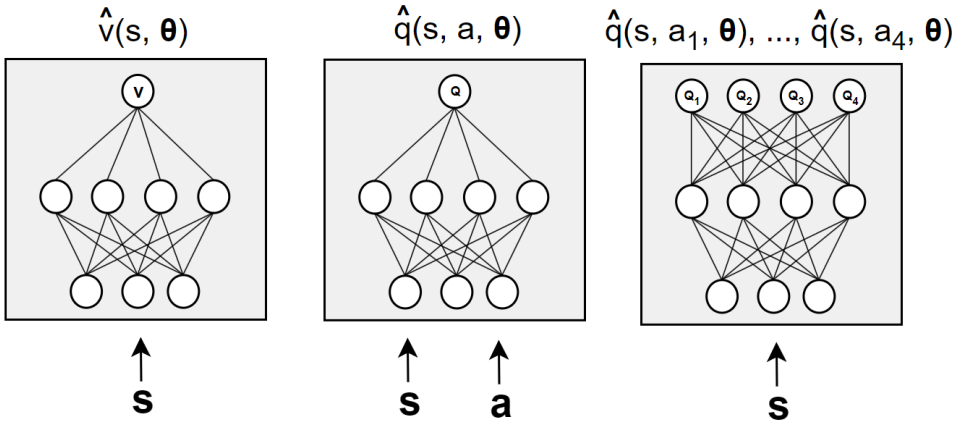

Figure 2: Three different ways in which a neural network may be used to approximate a value function [45].

Q-functions allow us to apply a form of online value iteration, known as q-learning, which needs only samples of state transitions, actions and rewards. The sampled transitions may be obtained from a real system or through simulation. Given a sample transition (s, a, r, τ, s'), q-learning updates the value $q(s, a)$ by [44];

$$q(s,a) \leftarrow q(s,a) + \alpha\Big(r + e^{-\beta\tau} \max_{a' \in \mathcal{A}_{s'}} q^{\star}(s', a') - q(s,a)\Big), \tag{3}$$

in which r is the observed time-dependent reward, τ is the observed time between consecutive decision epochs, and $0 < \alpha \leq 1$ is a learning rate.

One of the primary challenges in RL is to handle the curse of dimensionality, which arises due to the exponential growth in the number of possible states and actions. To overcome this issue, RL algorithms often rely on function approximation methods, such as neural networks or decision trees, to approximate the optimal value function or policy.

An approximate model for the optimal q-function is denoted by $\hat{q}(s, a; \boldsymbol{\theta})$, in which $\boldsymbol{\theta}$ is a set of parameters that must be trained on sampled data. Fig 2 illustrates how we can use a neural network to approximate a value function.

In the leftmost side in Fig. 2, the approximation involves providing the state as input to a neural network, which outputs the state-value for the given state. In the center form, both the state and action are given as input to the network, and the output is the action-value for the given state and action pair. Finally, in the rightmost form, only the state is provided as input, and the output is the action-value for all feasible actions given the state.

3.3 Discrete-event simulation

In order to train an agent, we must obtain samples from state transitions, actions and rewards. Although in principle we can use samples collected from interactions of the training agent with a real environment, this is generally inefficient and risky. Samples are often obtained by computer simulation. In this work, we obtain samples by using discrete-event simulation (DES).

DES is a powerful simulation paradigm which allows us to simulate discrete-state systems which evolve in continuous time [46–48]. DES has been successfully applied to queuing systems, transportation systems [49] and discrete-event dynamic systems in general.

DES exploits the fact that stochastic discrete-state systems remain at the same state for random finite time intervals and state transitions occur only at discrete points in time (events). A DES simulator keeps a record of random events in a list and jumps between events, updating system states and collecting statistics only at the times in which events occur. This greatly accelerates simulation and makes efficient use of computational resources.

4 Formulation of the dynamic vehicle dispatching problem as a semi-Markov decision process

In the following paragraphs, we formulate the process of dynamically dispatching vehicles to customers requests as a semi-Markov decision process. As such, we define the basic components of an SMDP: the environment, the state and action spaces, the reward function, and the environment dynamics.

4.1 Environment representation

We represent the environment as the space where trip requests can originate, and we use a simple coordinate system based on latitude and longitude to represent the location of vehicles and trips. Fig. 3 illustrates an example environment with a spatial distribution of vehicles and requests at a given time. It is important to notice that our model assumes complete knowledge of the environment, allowing the system to utilize information about all vehicles and requests at any time, such as locations and current status. Due to the high degree of digitalization of current vehicle dispatching systems, this is a realistic assumption.

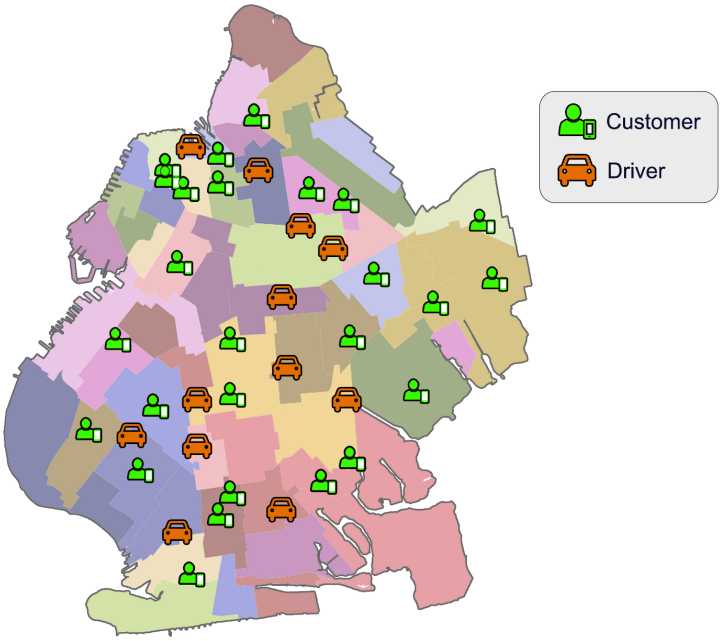

Figure 3: Spatial representation of the environment in the dynamic vehicle dispatching problem

4.2 Disadvantages of an MDP formulation

Most previous works in the literature model the DVDP as an MDP. At discrete-time decision epochs, a decision agent (e.g., a mobility software platform) observes the current state of the system and has to decide which vehicles will serve which requests. This involves solving a matching problem, whose set of possible solutions may be very large due to its combinatorial nature. Fig. 4 shows an example of how the DVDP is solved using a matching approach with an MDP framework. The time intervals between decision epochs in the MDP are of equal length, which can be a limitation in solving certain problems. For the DVDP, this characteristic of MDPs can result in two significant consequences. First, the system must wait for a fixed amount of time between consecutive decision epochs, which can result in multiple events occurring before the next decision epoch is triggered.

For example, at time $t = 1$ in Fig. 4, the system has $n = 5$ ride requests waiting and $m = 4$ vehicles available. In this case, there are $5! = 120$ possible matchings (assuming the existence of one dummy vehicle). In real-world problems, the number of possible solutions is $O(\max(n, m)!)$, making it hard to find the best (or even a good) decision in a timely manner. Finding the best matching corresponds to solving a generalized assignment problem, which is known to be

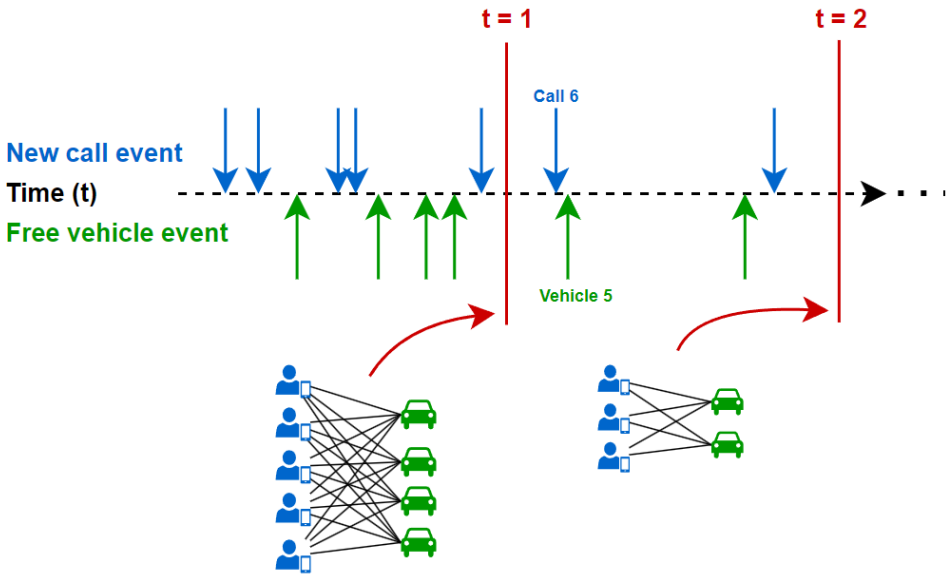

Figure 4: Decision epochs in the dynamic vehicle dispatching problem modeled as an MDP. Since decision epochs occur at fixed time intervals, the agent has to solve an NP-hard generalized assignment problem at each decision epoch.

NP-hard. The approaches based on MDP circumvent this difficulty by relaxing the full problem to a linear assignment problem, which can be solved efficiently, but ignores many constraints of the real problem.

A second implication of the MDP formulation is that the fixed time intervals between decision epochs also affect customers' waiting time. In the example depicted in Fig. 4, when vehicle 5 becomes free, there are 2 ride requests waiting for assignment. These ride requests need to wait until the second decision epoch at $t = 2$ before they can potentially receive an assignment. This can result in an increase of waiting time since vehicle 5 could have been assigned to request 6 as soon as it got free.

In summary, in real-world scenarios, the environment can be very large, and making optimal decisions can become very hard when using an MDP formulation.

4.3 Proposed event-based SMDP formulation

We notice that we can overcome many of the abovementioned disadvantages of the MDP formulation by adopting an event-based approach, which reduces the complexity of the assignment problem involved. We define two important events in the system, which trigger decision epochs: the rise of a new call (new call event) and the completion of a vehicle trip (new vehicle event).

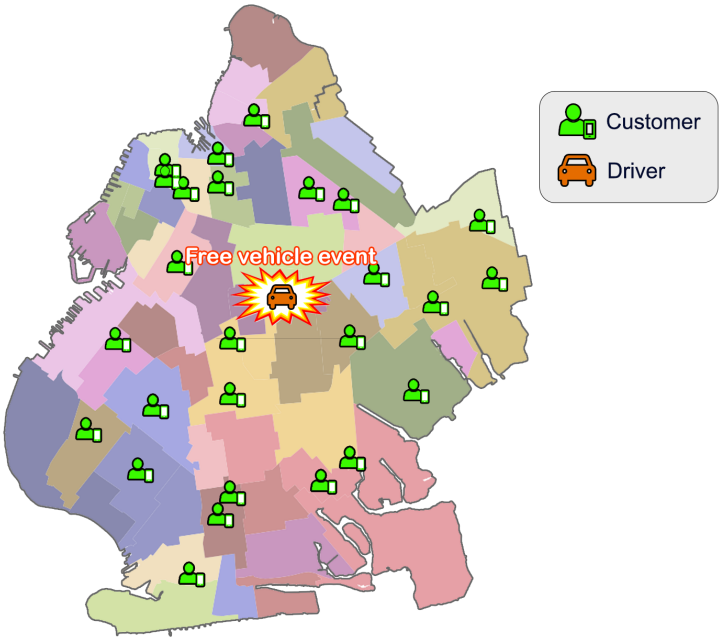

Figure 5: Spatial representation of a free vehicle event. All shown customers are waiting for service.

Figure 5 illustrates the free vehicle event. Notice that, in this case, we only need to decide which request to assign the vehicle from the pool of waiting requests. This assumes there are no other vehicles available at this same time, since in a scenario with more waiting requests than available vehicles, a free vehicle must have been assigned in a decision epoch before the current one. This results in a one-to-many assignment problem, which is simpler than the many-to-many assignment problems encountered when using an MDP approach. The same reduction in assignment complexity occurs when a new call event arises in the system, as illustrated in Figure 6. In this case, the problem is reduced to selecting the best vehicle to serve the new request.

Since the events occur stochastically over time, the time intervals between consecutive decision epochs are random. This requires us to model the DVDP as an SMDP. The first advantage of this formulation is that decisions are made as soon as possible, with no need to wait for a decision epoch at fixed time intervals. This also eliminates the problem of determining the length of the fixed time intervals between decision epochs. The second advantage is the reduction in decision complexity, which consistently transforms assignment problems into one-to-many configurations, as mentioned earlier because we focus on the entity that triggers the observed event. Fig. 7 shows how the decision epochs occur

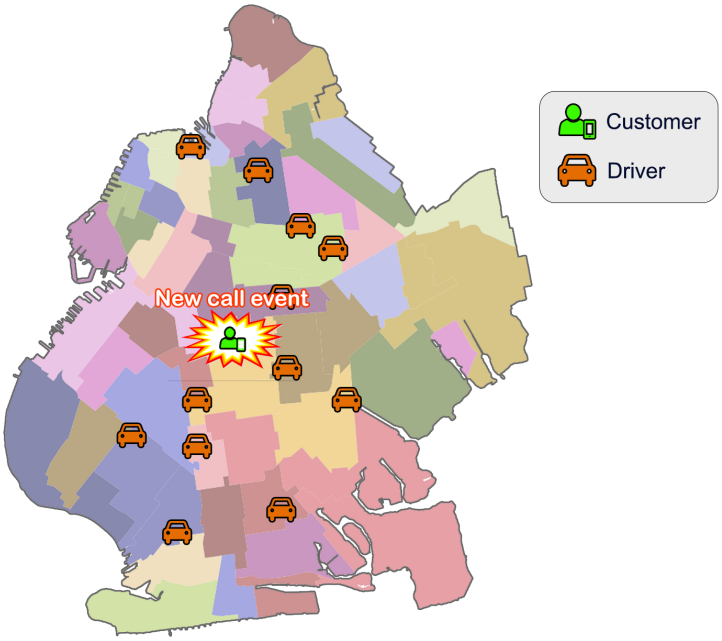

Figure 6: Spatial representation of new call event, in which one of the availables cars must be assigned to the call.

over time in our approach.

In Fig. 7, it is important to notice that the decision-making epochs, represented by the red dotted vertical lines, occur at irregular intervals in continuous time. By anticipating the decision-making moment, the system has the opportunity to establish a good assignment whenever possible, in contrast to the traditional MDP approach that requires waiting for a predetermined moment to make a decision. In particular, notice that vehicle 5 can be assigned to call 5 or call 6 as soon as it gets free (cf. Fig. 4), reducing their waiting times. Regarding the increase of decision epochs, this will not be a problem, as the assignment problems in this approach are substantially simpler and are solved quickly.

4.4 State and action spaces

At each decision epoch, the system state faced by the agent depends on the event triggered. For instance, in the event of a free vehicle, the state will be characterized by the specific vehicle, while the actions to be taken will be determined based on the pending requests. Conversely, when a new request (a new call) event occurs, the state will be defined in terms of the request, and the actions will be established by considering the available vehicles within the fleet. This adaptive

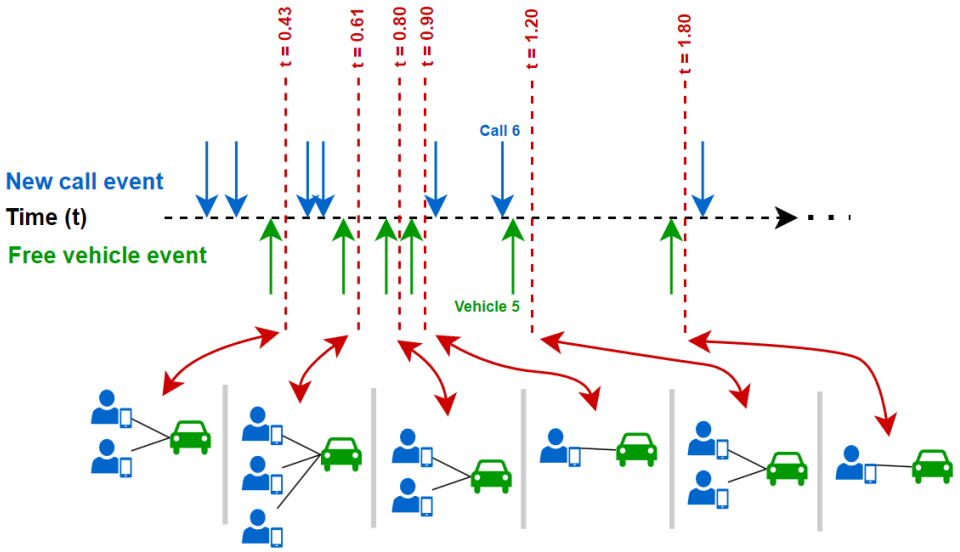

Figure 7: Decision epochs in the dynamic vehicle dispatching problem modeled as an MDP. Since decision epochs occur as soon as an event occurs, the agent solves a simple one-to-many assignment problem.

approach ensures that the data structure provided to the agents aligns with the specific event and enables them to make informed decisions based on the pertinent information.

Table 1 presents the seven features related to the vehicles. When the free vehicle event is triggered, these features represent the state. When the new call event is triggered, the state refers to the trip that arrives in the system. Table 2 shows the features related to incoming calls. Furthermore, in both data structures, we append some context features. Table 3 presents the three context features considered in this work.

The action space also depends on the type of event. In the case of a free vehicle event, the action space corresponds to the set of all waiting requests. The agent then chooses which request will be served by the current free vehicle (See Fig. 5). In the case of a new call event, the action space corresponds to the set of all currently available vehicles. The agent then chooses which vehicle will serve the new request (See Fig. 6). We emphasize the reduction of the action space, which encompasses only a one-to-many assignment, while an MDP formulation demands a hard many-to-many assignment.

Index	Feature	Description
1	X coordinate of the vehicle location	The X coordinate of the current vehicle location.
2	Y coordinate of the vehicle location	The Y coordinate of the current vehicle location.
3	X coordinate of the vehicle destination	The X coordinate corresponds to the destination of the vehicle's movement. In the case where the vehicle is not occupied, this coordinate is set to the same coordinate of the index 1.
4	Y coordinate of the vehicle destination	The Y coordinate corresponds to the destination of the vehicle's movement. In the case where the vehicle is not occupied, this coordinate is set to the same coordinate of the index 2.
5	Time to finish the service	Estimated time for the vehicle arrive at the destination of the call. In the case where the vehicle is not occupied, this feature is set to 0.
6	Cancel probability	The probability of the driver reject an assignment suggested by the system.
7	Vehicle status	This feature is set to 0 when the vehicle is idle and 1 when the vehicle is occupied.

Table 1: Vehicle state features

4.5 Reward function

The agents in this work receive rewards based on the duration of the trip and customer waiting time. The time segmentation of the service process used in this work is presented in Fig. 8. We denote the total service time as τ, which corresponds to the time interval between the assignment of a vehicle to a call and the arrival of the vehicle at the ride destination. τ is the sum of τ_p, the time interval between vehicle assignment and its arrival at the request origin, and τ_d, the time until the arrival at the ride destination.

The plain reward R associated with the agent assigning a vehicle to a request is defined as proportional to the estimated time of the ride:

$$R = \hat{\tau}_d + b,$$

in which $\hat{\tau}_d$ represents the estimated time it takes to travel from the origin to the destination of the ride request. The idea is that longer rides will provide higher monetary rewards to the drivers.

Index	Feature	Description
1	X coordinate of the call origin location	The X coordinate of the call origin location.
2	Y coordinate of the call origin location	The Y coordinate of the call origin location.
3	X coordinate of the call destination location	The X coordinate of the call destination location.
4	Y coordinate of the call destination location	The Y coordinate of the call destination location.
5	Time the ride request arrived in the system	The time that the customer sent the ride request to the system.

Table 2: Ride request features

Index	Feature	Description
1	Resource/demand ratio	Quantity of vehicle of the fleet divided by the quantity of new calls that arrives at the last 15 minutes. At the beginning of the process this is set to 1.
2	Week cyclical feature 1	Week time encoded with the following formula: $$\sin(2\pi(m/10080))$$ where m represents the minute of the week.
3	Week cyclical feature 2	Week time encoded with the following formula: $$\cos(2\pi(m/10080))$$ where m represents the minute of the week.

Table 3: Context features

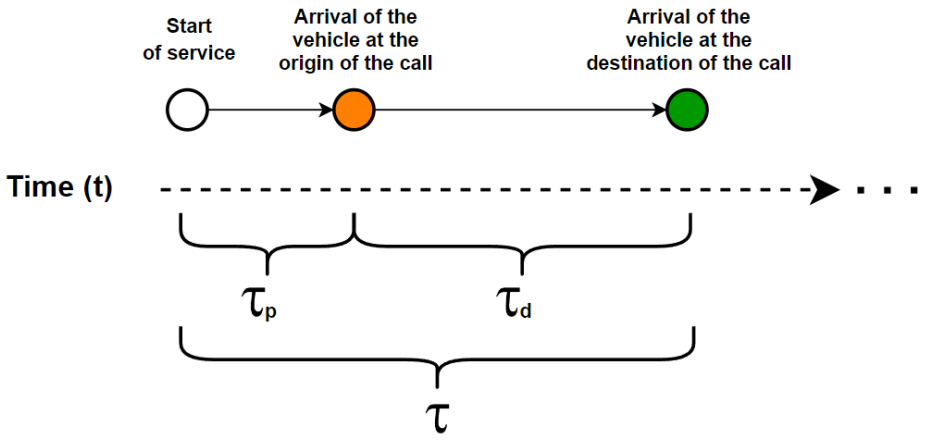

Figure 8: Times involved in the call handling process

However, we also want the agent to increase the rates of served requests, which will also indirectly reduce the waiting times and cancellation rates. We then added a constant term b, which acts as a fixed reward independent of the estimated duration of the ride $\hat{\tau}_d$. The parameter b represents a hyperparameter in our model that requires specification based on empirical data obtained from real-world applications. For instance, within the context of a ride-hailing platform, b may denote a fixed fare charged for each trip or can be determined through computational optimization. It is worth noting that a higher value of b encourages agents to undertake additional trips, thereby indirectly reducing the overall duration of individual trips. Conversely, a lower value of b provides incentives for agents to opt for longer trips, aiming to maximize the cumulative reward obtained.

In addition, due to the time-dependence of the problem, the reward signal that the agents receive is further discounted over time. For this, we use the accumulated discounted reward:

$$\begin{aligned} R_{disc} &= \frac{R}{\tau} + \gamma\frac{R}{\tau} + \gamma^2\frac{R}{\tau} + \cdots + \gamma^{\tau-1}\frac{R}{\tau} \\ &= \frac{R(\gamma^\tau - 1)}{\tau(\gamma - 1)}, \end{aligned}$$

where R is the plain reward, γ the discount factor and τ is the total service time, as shown in Fig. 8.

4.6 Environment dynamics

The environment dynamics evolves continuously over time. State transitions occur at discrete points in time. Thus, the environment may remain at a given state for a random amount of time. For example, position of vehicles make part of the state of the system, and we assume these change only at discrete points in time, such as when a vehicle arrives at a request's origin or destination.

The environment encompasses four main sources of randomness: Ride requests appear in the environment at random times and locations, the movement of vehicles between locations takes varying durations, drivers may refuse to serve a request offered by the decision agent with a given probability, and customers have an unknown probability distribution associated with their maximum tolerance time to wait.

The logic of the environment dynamics is embedded in a discrete-event simulation model, whose details are described in Section 5.2.

5 Proposed solution approach

Given the SMDP formulation of the DVDP, we could in theory use exact methods such as value iteration or policy iteration to obtain an optimal decision policy. However, the state space in the DVDP is too large and exact methods rely on enumerating all states, which makes exact methods computationally infeasible (the well known *curse of dimensionality*). Moreover, since the environment is very complex, we also do not have direct access to the conditional probability distributions of state transitions and can only sample the environment dynamics through a simulator.

To address these issues, we use model-free reinforcement learning techniques. We approximate the q-functions using deep neural networks which are trained from experiences sampled from the discrete-event simulator and use the double q-learning algorithm to adjust the weights of the networks. The subsequent sections outline our proposed solution approach.

5.1 General training scheme

Since in our SMDP formulation of the DVDP, decision epochs occur at two different types of events, we train two different decision agents: the `NewCallAgent` and the `FreeVehicleAgent`. We train our agents by combining real-world and simulated data. Our simulator utilizes the origin and destination locations and the arrival times of trips from real data. Additionally, we generate some simulated data using probability distributions, which will be discussed in the following sections.

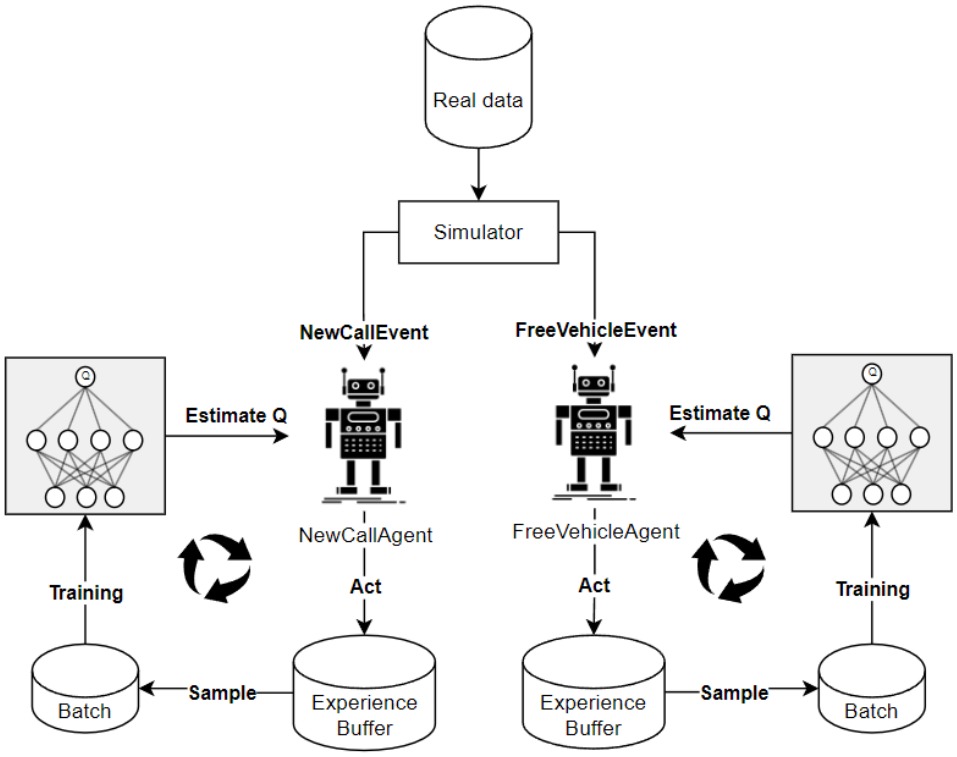

Figure 9: General training scheme of the two decision agents

Our agents make decisions based on q-value estimates, which measure the quality of a given action in a specific state. The q-values are the output of deep neural networks.

During the initial stage of the training process, the agents tend to make random decisions as they learn about the consequences of their actions in the environment. A sample transition is represented by a tuple

(current state, next state, action, reward, event duration).

As the simulation goes on, sample transitions are stored in a pool called the "experience buffer" (EB). Each agent has its neural network. To train these neural networks, we randomly sample a batch of experience tuples from the EB at every successful assignment and optimize their weights using the double Q-learning algorithm. Due to the random occurrence of decision epochs and the eventual subsequent assignments, the agents are trained simultaneously during the simulation process, without a predefined order. This general scheme is presented in Fig 9.

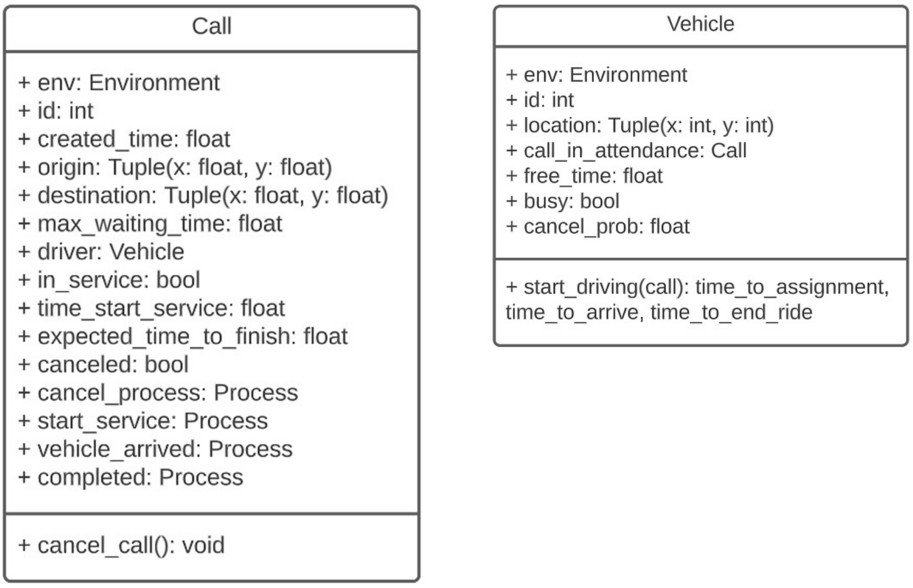

Figure 10: Class definitions of the basic entities call and vehicle. Squares in the top show class attributes while squares in the bottom show class methods.

5.2 Discrete-event simulator

To implement the simulation environment, we utilize DES, which aligns well with our event-based approach. We implement the simulator with the aid of SimPy, a DES library for the Python language.

5.2.1 Entities, agents and environment

We define three categories of objects which compose the simulator: basic entities, agents (decision makers) and the environment. The two diagrams displayed in Fig. 10 show the class definitions of the basic entities: calls and vehicles.

The `Call` class represents the ride requests made by customers. The main attributes are the time of creation, origin, destination, and maximum waiting time the customer is willing to wait to be served. `Call` objects are created during the course of a simulation by a function that based on real-world data draws their creation times, origins, and destinations and randomly defines the maximum waiting time.

The `cancel_call()` method of the `Call` class is a process function as defined in the SimPy library. Process functions are implemented as coroutines, which are special kinds of functions that can be interrupted and resumed at a later time

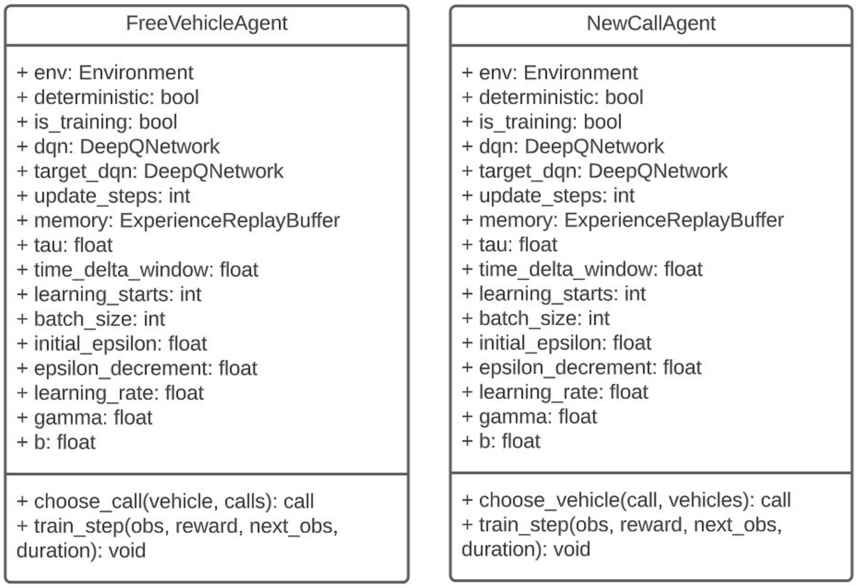

Figure 11: Class definitions of the two decision agents. Squares in the top show class attributes while squares in the bottom show class methods.

at the same point where it stopped executing. The `cancel_call()` method is started when a new object of the `Call` class is instantiated, and its objective is to monitor the maximum time that each specific customer can tolerate to wait. When the tolerance is reached, the `cancel_call()` method removes the call from the waiting calls pool.

The `Vehicle` class represent the vehicles in the simulator. The main attributes of a vehicle are its current location and busy status. Each vehicle is created at the start of a simulation and may serve many calls during a simulation run. The `start_driving()` method of the `Vehicle` class is also a SimPy process function. Is starts when a vehicle is assigned to a call and finishes when the vehicle arrives at the destination, when it triggers a `FreeVehicleEvent`.

Fig. 11 illustrates the classes used to represent the agents, which are the decision-makers of the system. Our implementation includes two decision-makers, one for each type of event. The two agent classes are very similar, differing mainly in their action sets. The `FreeVehicleAgent` class is responsible for choosing a call from the pool of waiting calls, while the `NewCallAgent` class is responsible for choosing a vehicle. It is important to note that each of these has two deep q-networks. It is because we utilize the double-DQN algorithm to reduce the problem of bias maximization. The b attribute in these classes refers to the hyperparameter of the reward function presented in the section 4. Most of the

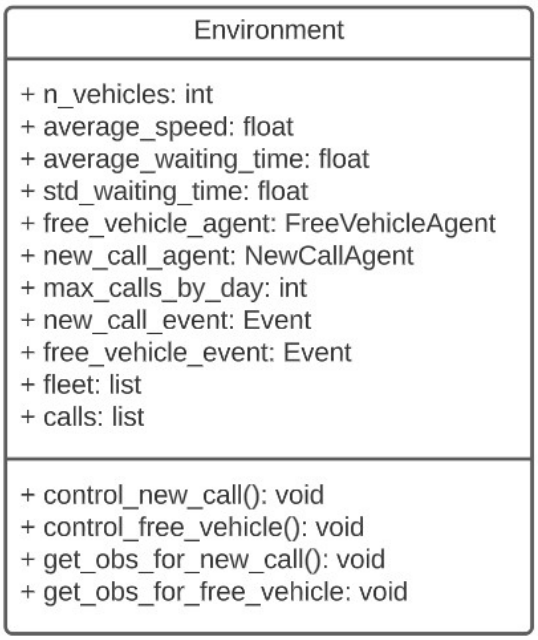

Figure 12: Environment

other parameters are related to the DQN algorithm and the neural networks involved.

The core of our simulator is the `Environment` class, which is presented in Figure 12. In addition to controlling the entire process flow, this class allows for the configuration of several simulation parameters, such as the number of vehicles in the fleet, the average speed of the vehicles, and the maximum number of daily calls. Additionally, the `Environment` class is responsible for providing state tuples to the agents through the `get_obs_for_new_call()` and `get_obs_for_free_vehicle()` methods.

5.2.2 Event-handling routines

A DES simulator works by maintaining a time-ordered list of events and processing the events by using specific event-handling routines. We describe below the two main routines in our simulator, which handle the `NewCallEvent` and `FreeVehicleEvent`.

When a `NewCallEvent` is triggered, the `NewCallAgent` immediately scans all vehicles in the fleet to determine the most suitable one for the assignment. It then uses a decision policy to choose a particular vehicle. If the chosen vehicle is free, the agent initiates the assignment process, which requires confirmation from both the driver and the customer. On the other hand, if the selected vehicle is

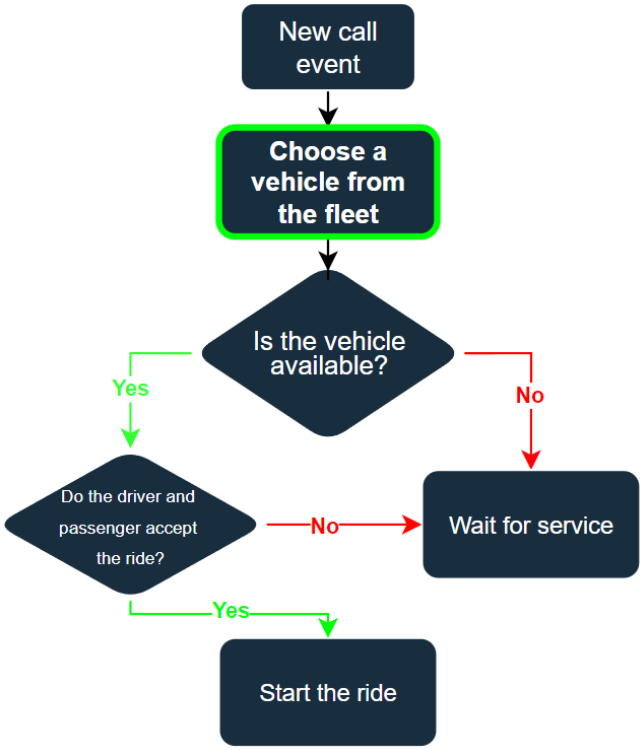

Figure 13: New call process flow

busy, the system places the call in a waiting call queue until it is assigned. It can occur, for instance, when a vehicle is finishing a trip near the call origin. Fig. 13 shows the process flow of the routine, which handles the `NewCallEvent`.

When a `FreeVehicleEvent` is triggered, the `FreeVehicleAgent` queries the pool of waiting requests (calls). If there are calls waiting, it chooses one of the calls according to a decision policy. Otherwise, it does nothing and waits. Fig. 14 details the event-handling routines.

It should be noted that the `FreeVehicleAgent` will only need to decide if there is at least one call in the waiting calls pool. An important step in the flow is when the system suggests an assignment and must wait for the acceptance of both the driver and the customer. If both parties accept, the ride begins and the flow proceeds normally. However, in the occurrence of rejection by either party, the system will put the call to the waiting calls pool, and the driver will receive a recommendation from the `FreeVehicleAgent` to either reposition herself to the call location or await further instructions. In either scenario, the `FreeVehicleAgent` will wait for a maximum of 5 minutes until the vehicle is assigned to another call. If the vehicle remains unoccupied at the end of this

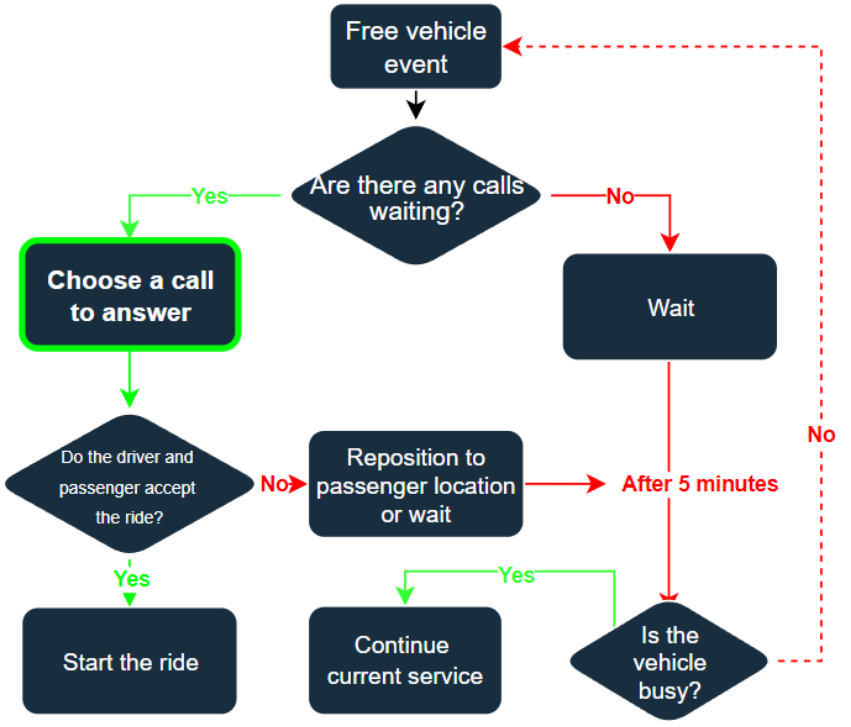

Figure 14: Free vehicle process flow

period, a new `FreeVehicleEvent` will be triggered, and the flow will continue.

It is worth noting that the assignment of a vehicle to a call suggested by the agents is only a proposal for both the customer and the driver. Hence we consider the possibility of rejection by either party. To simulate the potential for rejection by either party, we employed two probability distributions in our modeling approach. The first distribution characterizes the probability of rejection by the driver, and it is modeled using a beta density function. For each vehicle, at its creation time, we sample a probability p from the beta density, which represents the probability that the driver will reject an assignment. This implies that each vehicle is associated with a unique probability of rejecting an assignment, and when an agent attempts to make an assignment of the vehicle, a Bernoulli trial is performed using this probability. It is important to note that in a real-world application, this probability can be estimated by analyzing historical data on assignment rejections.

We employ an additional distribution that characterizes the customers' tolerance for waiting. Specifically, we adopt a gamma probability density to model this. During the call creation process, each call is assigned a maximum waiting time, which is randomly sampled from the gamma density. Consequently, when

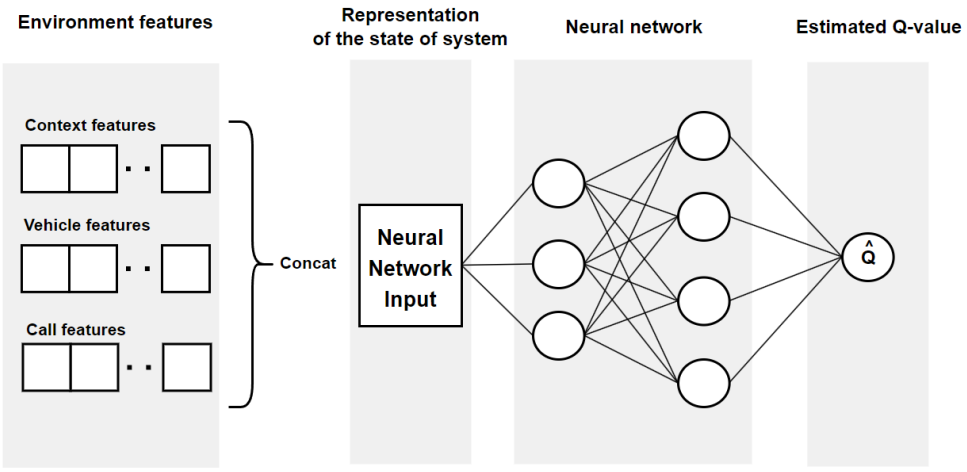

Figure 15: Data flow in the q-value estimation process

an agent proposes an assignment, customers will reject it if the estimated time for the vehicle to arrive at the pickup location exceeds the previously sampled maximum waiting time for the call.

5.3 Agents' brains: deep neural networks

Our intelligent agents are driven by deep neural networks which estimate q-values of current state and actions. In the case of the `NewCallAgent`, the action corresponds to assigning one of the available vehicles to the newly arrived call, while the action of the `FreeVehicleAgent` corresponds to assigning the free vehicle to one of the waiting calls. The agents decide on the best actions by querying their neural networks and identifying the action with the maximum q-value. Notice that as the agents perform one-to-many assignments, the number of possible actions is considerably smaller than when performing many-to-many assignments.

Fig. 15 schematically shows the input and output of the deep neural networks. The input is the concatenation of the tuples presented in Section 4 and the output it the corresponding estimate of the q-value, which represents the quality of the action. Each concatenated tuple represents a potential assignment between a vehicle and a call in a specific context. The agents receive several concatenated tuples and identify which one has the highest q-value. Notably, the list of tuples passed as input to the neural networks of the `FreeVehicleAgent` contains the same vehicle attributes and different call parameters, while for the `NewCallAgent`, the lists contain the same call attributes and different vehicle features.

To mitigate the overestimation bias inherent in traditional q-learning training

[50], we apply the double q-learning algorithm [51]. This algorithm is an extension of the q-learning that addresses the overestimation issue caused by the maximum operator in its update formula, as shown in Eq. (3). The proposed strategy in double q-learning is to introduce two distinct q-value functions, identified below by letters A and B, in the updating equation:

$$q^A(s,a) \leftarrow q^A(s,a) + \alpha\Big(r + e^{-\beta\tau} q^B(s', \underset{a' \in \mathcal{A}_{s'}}{\operatorname{argmax}}\, q^A(s',a')) - q^A(s,a)\Big).$$

Notice that, rather than using a single q-value function for both action selection and evaluation, double q-learning separates the functions, assigning one q-value function for action selection and another for action evaluation.

Since double q-learning updates two different q-values simultaneously, in our implementation each of the two agents have two deep neural networks dedicated to estimating the q-values, labeled network A and B. Then, given a sample transition (s, a, s', r) at iteration t, the weights of network A are updated according to

$$\boldsymbol{\theta}^{\mathrm{A}}_{t+1} \leftarrow \boldsymbol{\theta}^{\mathrm{A}}_{t} + \alpha(Y^{\mathrm{DoubleQ}} - q(s,a;\boldsymbol{\theta}^{\mathrm{A}}_{t}))\nabla_{\boldsymbol{\theta}^{\mathrm{A}}_{t}} q(s,a;\boldsymbol{\theta}^{\mathrm{A}}_{t}),$$

which is essentially a gradient descent step. The target is given by [52]:

$$Y^{\mathrm{DoubleQ}} = r + e^{-\beta\tau} q\Big(s', \underset{a' \in \mathcal{A}_{s'}}{\operatorname{argmax}}\, q(s',a';\boldsymbol{\theta}^{\mathrm{A}}_{t});\boldsymbol{\theta}^{\mathrm{B}}_{t}\Big), \tag{4}$$

in which $\boldsymbol{\theta}^{\mathrm{A}}_{t}$ denotes the current parameters of neural network A, while $\boldsymbol{\theta}^{\mathrm{B}}_{t}$ represents the current parameters of neural network B. Network B is updated similarly, just swaping letters A and B in Eq. (4).

The learning process begins after the EB has accumulated a certain number of samples (controlled by the parameter `learning_starts`, shown in Fig. 11). We perform gradient steps using mini-batches sampled at each successful assignment in the environment. It is important to note that both neural networks have the same structure, except for their parameters. In one neural network, we perform gradient descent at every step, while in the other, we set the parameter `update_steps` to control the number of steps before the network parameters are synchronized.

5.4 Computational implementation

Besides the abovementioned use of SimPy, we also use other Python libraries to implement the simulator and neural networks. Python 3.7.13 is utilized for the entire computational implementation of this work. Several packages are necessary, and the most important ones with corresponding versions are presented in the Table 4.

Package	Version
NumPy	1.21.6
Pandas	1.3.5
GeoPandas	0.10.2
Simpy	4.0.1
Matplotlib	3.5.1
PyTorch	1.8.2

Table 4: Python packages used in the computational implementation

6 Numerical results

In our experiments, we utilized the for-hire vehicle trip records dataset provided by NYC Taxi and Limousine Commission[1] as input to the simulator. Specifically, we used the data from January 2022 for training our agents and February 2022 data to evaluate their performance. To restrict the scope of our study, we filtered the trips and included only those that occurred in Brooklyn and those completed through the Uber or Lyft platform. Distances were computed using the Manhattan distance.

6.1 Training phase

In the training phase, we set the discount factor for the agents to $\gamma = 0.9$ and the replay buffer size to 20,000. Additionally, we set the parameter `update_steps` to synchronize the target and behavior networks at every 10,000 training steps. During training, we utilized an ϵ-greedy policy with $\max \epsilon = 1$, $\min \epsilon = 0.05$, and an ϵ-decrement of 0.99995. The agents started training once the replay buffer was half-full, which means that training started when the replay buffer amounted to at least 10,000 experience tuples available.

The q-networks were constructed with two fully connected hidden layers and leaky-ReLU activation functions. The first hidden layer has 64 neurons, and the second has 32 neurons. The batch size is 32 and the learning rate is set to 0.001. We used Adam as optimization algorithm and used as loss function the smooth L_1 loss.

In order to train the agents to handle various situations, we defined four scenarios based on the ratio of fleet size in relation to the number of daily calls. For the very easy scenario, we set the fleet size to 3% of the number of daily calls; for the easy scenario, we set it to 2%; for the medium scenario, we set it to 1%;

[1] https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

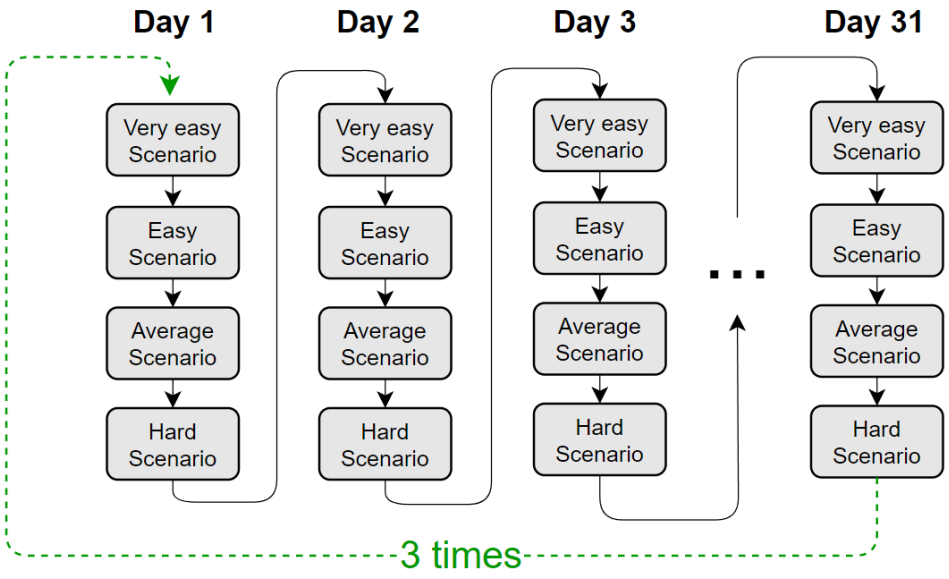

Figure 16: During the training phase, we iterate the 4 scenarios 3 times for each of the days of January 2022.

and for the hard scenario, we set it to 0.5%.

During the training phase, we set the maximum number of daily calls to 1,000 and we iterated 12 times on each day of the month, with 3 iterations for each scenario, as illustrated in Fig. 16. This leads to a total of 372 training days with each day having 24 hours. To evaluate the computational cost, we performed the training phase 10 times. Our findings showed an average duration of 34.63 minutes with a standard deviation of 1.05 minutes. The training was executed on a machine equipped with an Intel Core i9 13900k processor, 64 GB of memory running at 5600 MHz, and an RTX 4090 graphic card. During the training phase, the machine was solely dedicated to executing this task.

The average reward achieved during the training phase of the agents is presented in Fig. 17. The average reward values shown were computed at intervals of every 1,000 rewards received. The evolution of average action values during the training phase for both agents is shown in Fig. 18. These average values were computed for every 1,000 chosen actions. The convergence of neural networks is illustrated in Fig. 19, which show the decrease in the loss function. These plots demonstrate the rapid approximation of the value function by the neural networks during the training process.

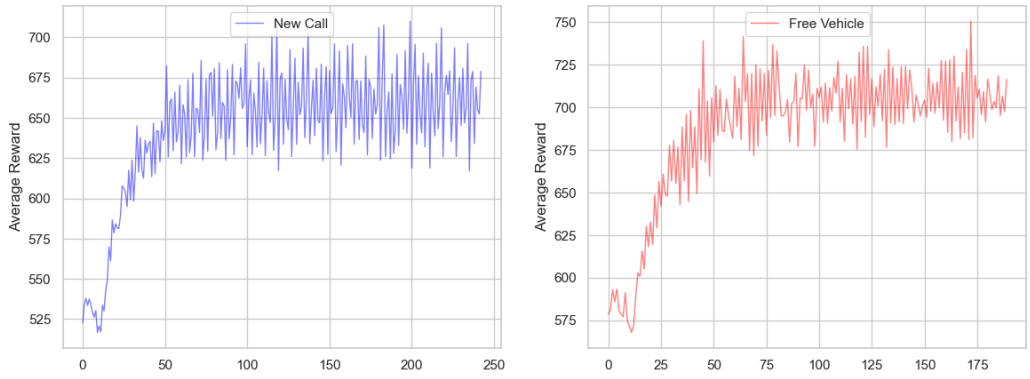

Figure 17: Average reward computed at intervals of 1,000 rewards received by each of the `NewCallAgent` and `FreeVehicleAgent` during the training phase

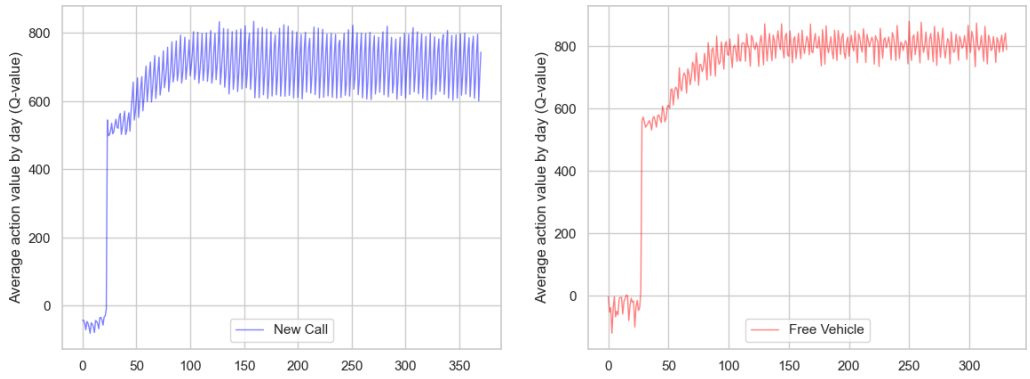

Figure 18: Average action values for every 1,000 chosen actions during the training phase

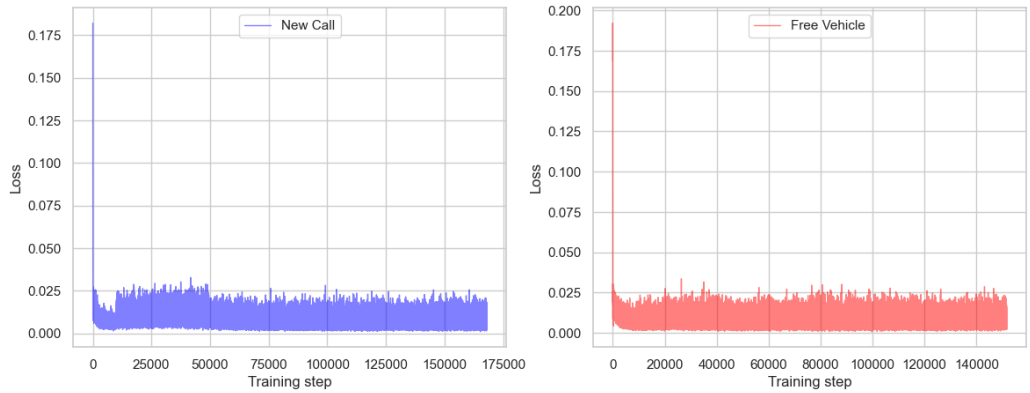

Figure 19: Loss function of the neural networks of both agents during the training phase

6.2 Testing phase

During the testing phase, we set the number of daily calls to 100,000, surpassing the overall daily demand of Uber in Brooklyn. We conducted tests during entire month of February 2022. Notice that this data were kept fully separated from the data used during training, so that there is no data leakage and they were completely new to the trained agents.

We used as benchmarks to the trained policy four baseline policies, which are frequently used in practice due to their simplicity and easiness of implementation:

1. First In First Out (FIFO): The agent always chooses the longest waiting call to assign a vehicle.

2. Last In First Out (LIFO): The agent always chooses the most recent call to assign a vehicle.

3. Nearest Neighbor (NN): The agent assigns the vehicle closest in distance to a new arriving call or the closest waiting call when a vehicle gets free.

4. Random: The agent chooses randomly a call or a vehicle at decision times.

We compare policies according to three performance measures:

1. Average delay: The average time taken by vehicles to reach the call origin. This time is measured from the moment the call is created until the vehicle arrives at the call origin.

2. Cancellation rate: The ratio of canceled calls to the total number of calls received in a day.

3. Total service time: The sum of the distances traveled from the origin to the destination for all completed trips in a day.

Fig. 20 presents the average daily delay of our agents, referred to as DQN, along with the baseline policies, during the testing phase. It is important to note that none of the policies had prior exposure to the demand in this scenario. Fig. 21 illustrates the average daily cancellation rate while Fig. 22 displays the average daily total service time of all policies.

7 Discussion

The plots shown in Figs. 17, 18, and 19 demonstrate the fast convergence of our training method. The average q-values exhibit consistently low variance over

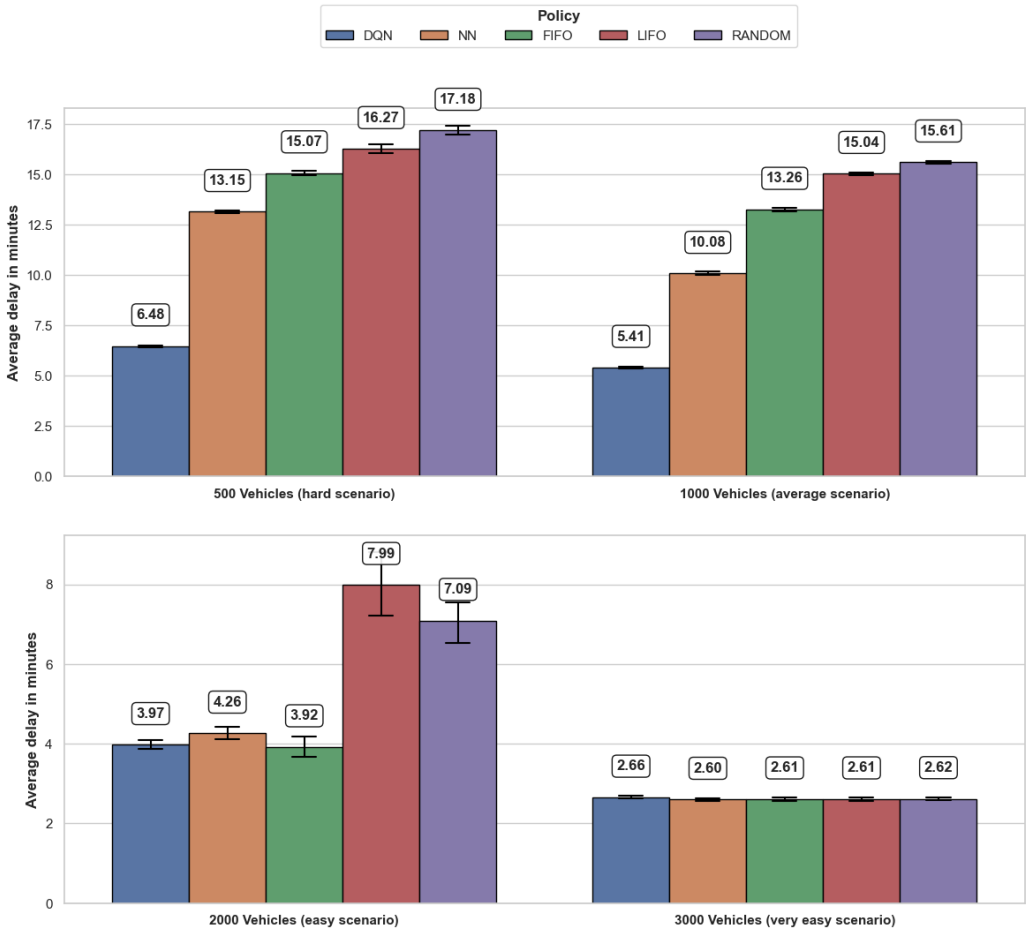

Figure 20: Average delay (in min.) during the test phase. Each bar represents the average daily delay over the days of February 2022. The horizontal lines at the top of bars represent 95% confidence intervals.

time, especially for the free vehicle event. On the other hand, the average rewards of the agent responsible for this event display some instability during training, but in the long term the average rewards become acceptable. Additionally, it is noticeable that both the rewards and q-values do not exhibit significant increases after a certain point in the training phase, suggesting that a shorter training duration may be sufficient to obtain results with similar performance measures.

Based on the findings depicted in Fig. 20, a clear advantage of our agents over the baseline policies becomes evident in the challenging and moderately challenging scenarios. In the hard scenario we achieved a reduction of approximately 50% and in the average scenario a reduction of 46% in average delay when compared with the second best policy (NN). However, this advantage is not so prominent

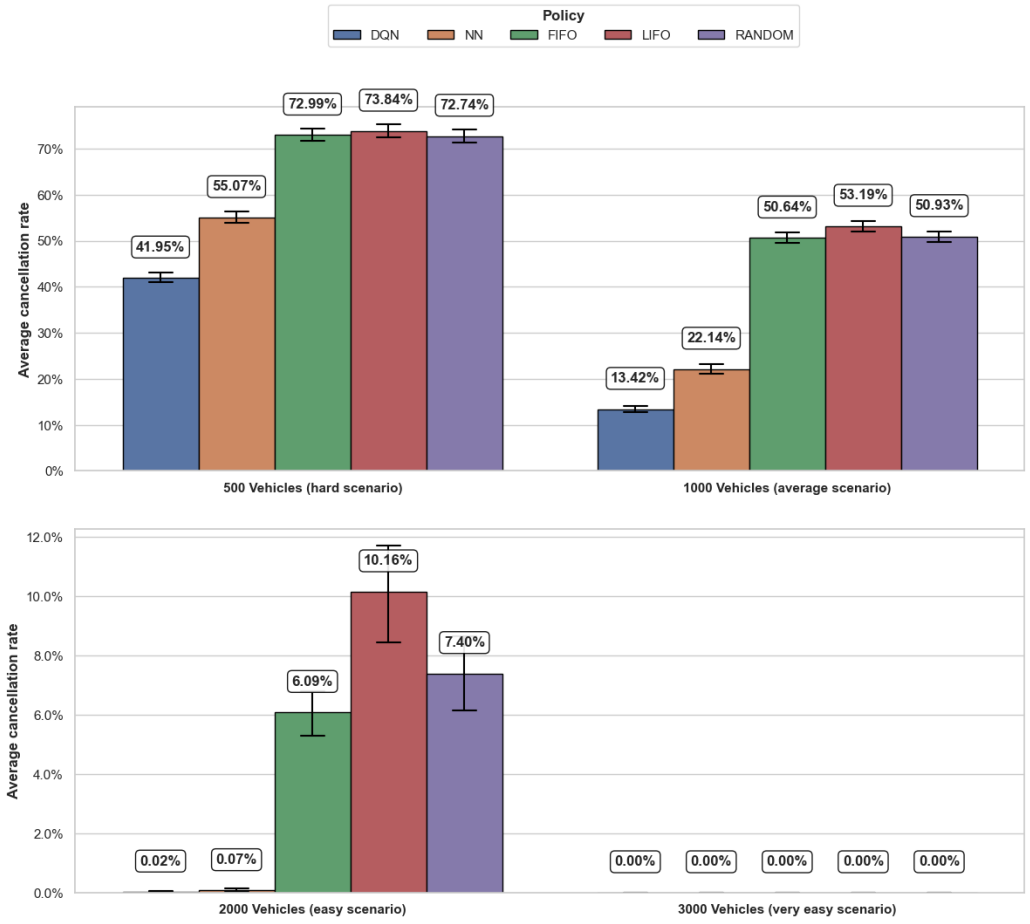

Figure 21: Cancellation rate during the test phase. Each bar represents the average daily cancellation rate over the days of February 2022. The horizontal lines at the top of bars represent 95% confidence intervals.

in the less demanding scenarios. Specifically, in the easy scenario, the delay achieved by the DQN approach is minimal, yet the NN and FIFO policies also demonstrate competitive performance in this regard. Furthermore, these three policies achieved narrow confidence intervals, indicating consistent results. In the scenario characterized as "very easy", wherein the number of drivers significantly surpasses the demand, all policies exhibit comparable performance. This outcome arises from the substantial pool of available drivers in this scenario, resulting in extensive spatial coverage of the map and minimization of the probability of a lack of nearby vehicles for new ride requests.

Fig. 21 also provides evidence of the superiority of the policy learned by our agents compared to the alternative policies. Notably, in the hard and average

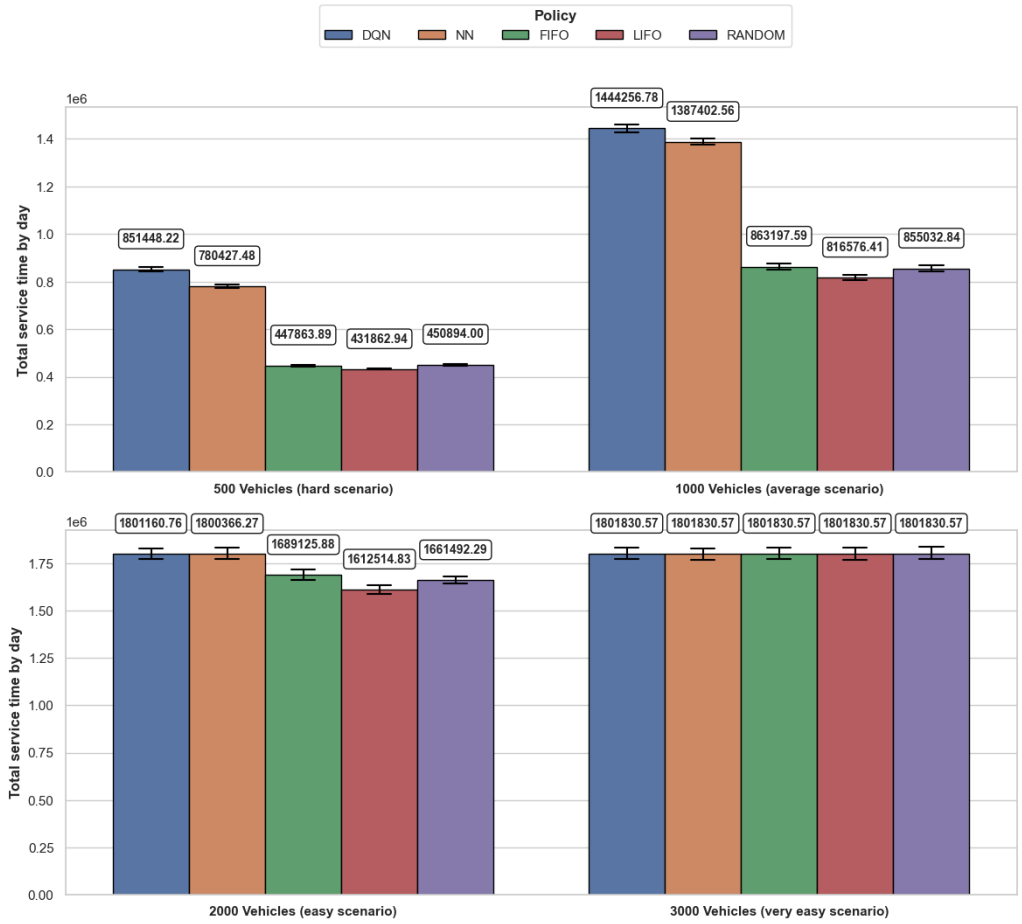

Figure 22: Total service time during the test phase. Each bar represents the average daily service time over the days of February 2022. The horizontal lines at the top of bars represent 95% confidence intervals.

scenarios, our approach achieved a significant reduction of approximately 13.12% and 8.72%, respectively, in the cancellation rate. This reduction can have a profound impact on mitigating overall customer dissatisfaction, indicating the effectiveness of our learned policy in minimizing cancellations. In the easy scenario, the NN policy exhibits exceptional performance, comparable to our approach. It is also worth noticing that, as can be seen in Fig. 20, the FIFO policy exhibits comparable performance to our DQN policy. On the other hand, in Fig. 21 we can see that FIFO exhibits higher cancellation rate than our DQN policy. As an aside, in the very easy scenario, all policies achieve perfect performance due to the abundant availability of vehicles, as in the previous analysis of average delay.

Finally, we consider the total service time performance measure. Total service

time is a proxy for total revenue of drivers, since the more time drivers are servicing customers the higher their revenues. As illustrated in Figure 22, our approach consistently surpasses all baseline policies in both the hard and average scenarios, corroborating the trends observed in previously analyzed performance measures. In these scenarios, the NN policy emerges as the second-best alternative, demonstrating its efficacy as a myopic policy. In the easy and very easy scenarios, all policies exhibit similar performance, with the LIFO policy being the least favorable. Notably, in the easy scenario, our approach and the NN policy achieve comparable performance, resulting in a technical draw. In the very easy scenario, all policies demonstrate flawless performance, due to the absence of cancellations, as evidenced by Figure 21. Consequently, maximum total service time is achieved across all policies in the very easy scenario.

8 Conclusions

In this paper, we proposed a new event-based approach for the dynamic vehicle dispatching problem. We formulated the problem as a semi-Markov decision process and developed a solution approach by integrating a discrete-event simulator with deep reinforcement learning. To this end, we employed the double deep q-learning algorithm to train two different agents corresponding to new call and free car events. Agents' policies were approximated by deep neural networks. Our developed discrete-event simulator incorporates often overlooked characteristics in existing literature, such as the probability of a driver rejecting an assignment proposed by the system and the maximum waiting time tolerance of customers. We carried out extensive experiments that closely mirror real-world conditions with the use of real data from the city of New York and compared with alternative heuristic policies often used in practice.

The decision policy obtained through our proposed approach achieved a substantial reduction of up to 50% in the average customer waiting time relative to the second-best policy corresponding to assigning the nearest vehicle. Moreover, our approach also resulted in the lowest average cancellation rates and highest total service time, a proxy for total revenues received by drivers. Notice that achieving low cancellation rates is critical, since it mitigates potential customer disappointments, discouraging customers from seeking alternative services and promoting customer loyalty. Additionally, the increase in total service time contributes to elevated driver satisfaction within the mobility platform, since it results in more revenue and less idle time, leading to a more productive and fulfilling experience for them.

In future works, we aim to extend our approach by employing alternative RL algorithms and comparing their performance against recently proposed matching

strategies. Additionally, we plan to enhance the capabilities of our simulator by incorporating real-time traffic information. Finally, we intend to explore the potential of multi-agent approaches.

Acknowledgments

Funding: This study was financed in part by Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq; Grant No.: 407466/2021-5). We also thank NVIDIA Corporation for the GPU support.

References

[1] Shih-Fen Cheng and Xin Qu. A service choice model for optimizing taxi service delivery. In *2009 12th International IEEE Conference on Intelligent Transportation Systems*, pages 1–6, St. Louis, October 2009. IEEE. ISBN 978-1-4244-5519-5. doi: 10.1109/ITSC.2009.5309520.

[2] Zhiwei (Tony) Qin, Xiaocheng Tang, Yan Jiao, Fan Zhang, Zhe Xu, Hongtu Zhu, and Jieping Ye. Ride-Hailing Order Dispatching at DiDi via Reinforcement Learning. *INFORMS Journal on Applied Analytics*, 50(5):272–286, September 2020. ISSN 2644-0865, 2644-0873. doi: 10.1287/inte.2020.1047.

[3] Nicole Sadowsky and Erik Nelson. The Impact of Ride-Hailing Services on Public Transportation Use: A Discontinuity Regression Analysis. 2017.

[4] Guoju Gao, Mingjun Xiao, and Zhenhua Zhao. Optimal Multi-taxi Dispatch for Mobile Taxi-Hailing Systems. In *2016 45th International Conference on Parallel Processing (ICPP)*, pages 294–303, Philadelphia, PA, USA, August 2016. IEEE. ISBN 978-1-5090-2823-8. doi: 10.1109/ICPP.2016.41.

[5] Warren B Powell. A stochastic formulation of the dynamic assignment problem, with an application to truckload motor carriers. *Transportation Science*, 30(3):195–219, 1996.

[6] Hugo P. Simão, Jeff Day, Abraham P. George, Ted Gifford, John Nienow, and Warren B. Powell. An approximate dynamic programming algorithm for large-scale fleet management: A case application. *Transportation Science*, 43(2):178–197, 2009. doi: 10.1287/trsc.1080.0238.

[7] Gregory A. Godfrey and Warren B. Powell. An adaptive dynamic programming algorithm for dynamic fleet management, i: Single period travel times. *Transportation Science*, 36(1):21–39, 2002. doi: 10.1287/trsc.36.1.21.570.

[8] Ziqi Liao. Taxi dispatching via global positioning systems. *IEEE Transactions on Engineering Management*, 48(3):342–347, 2001.

[9] Der-Horng Lee, Hao Wang, Ruey Long Cheu, and Siew Hoon Teo. Taxi dispatch system based on current demands and real-time traffic conditions. *Transportation Research Record*, 1882(1):193–200, 2004.

[10] Kiam Tian Seow, Nam Hai Dang, and Der-Horng Lee. A collaborative multiagent taxi-dispatch system. *IEEE Transactions on Automation science and engineering*, 7(3):607–616, 2009.

[11] Lingyu Zhang, Tao Hu, Yue Min, Guobin Wu, Junying Zhang, Pengcheng Feng, Pinghua Gong, and Jieping Ye. A taxi order dispatch model based on combinatorial optimization. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 2151–2159, 2017.

[12] Dimitris Bertsimas, Patrick Jaillet, and Sébastien Martin. Online vehicle routing: The edge of optimization in large-scale applications. *Operations Research*, 67(1):143–162, 2019.

[13] Zhe Xu, Zhixin Li, Qingwen Guan, Dingshui Zhang, Qiang Li, Junxiao Nan, Chunyang Liu, Wei Bian, and Jieping Ye. Large-scale order dispatch in on-demand ride-hailing platforms: A learning and planning approach. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 905–913, 2018.

[14] Zhaodong Wang, Zhiwei Qin, Xiaocheng Tang, Jieping Ye, and Hongtu Zhu. Deep reinforcement learning with knowledge transfer for online rides order dispatching. In *2018 IEEE International Conference on Data Mining (ICDM)*, pages 617–626. IEEE, 2018.

[15] Enming Liang, Kexin Wen, William HK Lam, Agachai Sumalee, and Renxin Zhong. An integrated reinforcement learning and centralized programming approach for online taxi dispatching. *IEEE Transactions on Neural Networks and Learning Systems*, 2021.

[16] Xiaocheng Tang, Zhiwei Qin, Fan Zhang, Zhaodong Wang, Zhe Xu, Yintai Ma, Hongtu Zhu, and Jieping Ye. A deep value-network based approach for multi-driver order dispatching. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 1780–1790, 2019.

[17] Zhiwei Qin, Xiaocheng Tang, Yan Jiao, Fan Zhang, Zhe Xu, Hongtu Zhu, and Jieping Ye. Ride-hailing order dispatching at DiDi via reinforcement learning. *INFORMS Journal on Applied Analytics*, 50(5):272–286, 2020.

[18] Fei Miao, Shuo Han, Shan Lin, John A Stankovic, Desheng Zhang, Sirajum Munir, Hua Huang, Tian He, and George J Pappas. Taxi dispatch with real-time sensing data in metropolitan areas: A receding horizon control approach. *IEEE Transactions on Automation Science and Engineering*, 13 (2):463–478, 2016.

[19] John Holler, Risto Vuorio, Zhiwei Qin, Xiaocheng Tang, Yan Jiao, Tiancheng Jin, Satinder Singh, Chenxi Wang, and Jieping Ye. Deep reinforcement learning for multi-driver vehicle dispatching and repositioning problem. In *2019 IEEE International Conference on Data Mining (ICDM)*, pages 1090–1095. IEEE, 2019.

[20] Nicholas D Kullman, Martin Cousineau, Justin C Goodson, and Jorge E Mendoza. Dynamic ride-hailing with electric vehicles. *Transportation Science*, 2021.

[21] Zhidan Liu, Jiangzhou Li, and Kaishun Wu. Context-aware taxi dispatching at city-scale using deep reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems*, 2020.

[22] Xiaocheng Tang, Fan Zhang, Zhiwei Qin, Yansheng Wang, Dingyuan Shi, Bingchen Song, Yongxin Tong, Hongtu Zhu, and Jieping Ye. Value function is all you need: A unified learning framework for ride hailing platforms. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pages 3605–3615, 2021.

[23] E.V. Denardo. *Dynamic Programming: Models and Applications*. Dover Books on Computer Science. Dover Publications, 2012. ISBN 9780486150857.

[24] M.L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley Series in Probability and Statistics. Wiley, 2014. ISBN 9781118625873.

[25] R.S. Sutton and A.G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[26] Richard Bellman. *Dynamic Programming*. Princeton Univ. Pr, Princeton, NJ, 1957. ISBN 978-0-691-07951-6.

[27] Ronald A Howard. Dynamic programming and markov processes. 1960.

[28] Francois d'Epenoux. A probabilistic production and inventory problem. *Management Science*, 10(1):98–108, 1963.

[29] Alan S Manne. Linear programming and sequential decisions. *Management Science*, 6(3):259–267, 1960.

[30] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. *The Journal of Machine Learning Research*, 17(1):1334–1373, 2016.

[31] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, February 2015. ISSN 0028-0836, 1476-4687. doi: 10.1038/nature14236.

[32] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1-2):181–211, August 1999. ISSN 00043702. doi: 10.1016/S0004-3702(99)00052-1.

[33] Andrew G Barto and Sridhar Mahadevan. Recent advances in hierarchical reinforcement learning. *Discrete event dynamic systems*, 13(1-2):41–77, 2003.

[34] D. Bertsekas. *Dynamic Programming and Optimal Control: Volume II; Approximate Dynamic Programming*. Athena Scientific optimization and computation series. Athena Scientific, 2012. ISBN 9781886529441.

[35] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

[36] S. Gu, E. Holly, T. Lillicrap, and S. Levine. Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3389–3396, 2017. doi: 10.1109/ICRA.2017.7989385.

[37] Puze Liu, Kuo Zhang, Davide Tateo, Snehal Jauhri, Zhiyuan Hu, Jan Peters, and Georgia Chalvatzaki. Safe reinforcement learning of dynamic high-dimensional robotic tasks: Navigation, manipulation, interaction, 2023.

[38] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529, 2015.

[39] Anselmo R Pitombeira-Neto and Arthur HF Murta. A reinforcement learning approach to the stochastic cutting stock problem. *EURO Journal on Computational Optimization*, 10:100027, 2022. doi: https://doi.org/10.1016/j.ejco.2022.100027.

[40] Vladimir Samsonov, Karim Ben Hicham, and Tobias Meisen. Reinforcement learning in manufacturing control: Baselines, challenges and ways forward. *Engineering Applications of Artificial Intelligence*, 112:104868, 2022. ISSN 0952-1976. doi: https://doi.org/10.1016/j.engappai.2022.104868.

[41] Xin Jin, Zhentang Duan, Wen Song, and Qiqiang Li. Container stacking optimization based on deep reinforcement learning. *Engineering Applications of Artificial Intelligence*, 123:106508, 2023. ISSN 0952-1976. doi: https://doi.org/10.1016/j.engappai.2023.106508.

[42] W.B. Powell. *Approximate Dynamic Programming: Solving the Curses of Dimensionality*. Wiley, 2nd edition, 2011.

[43] D.P. Bertsekas and J.N. Tsitsiklis. *Neuro-dynamic Programming*. Athena Scientific, 1996.

[44] Steven Bradtke and Michael Duff. Reinforcement learning methods for continuous-time markov decision problems. *Advances in neural information processing systems*, 7, 1994.

[45] David Silver. Lectures on reinforcement learning. URL: `https://www.davidsilver.uk/teaching/`, 2015.

[46] A.M. Law. *Simulation Modeling and Analysis*. McGraw-Hill Education, 2014. ISBN 9780073401324.

[47] Abhijit Gosavi. *Simulation-Based Optimization*, volume 55 of *Operations Research/Computer Science Interfaces Series*. Springer US, Boston, MA, 2015. ISBN 978-1-4899-7490-7 978-1-4899-7491-4. doi: 10.1007/978-1-4899-7491-4.

[48] S.M. Ross. *Simulation*. Elsevier Science, 2022. ISBN 9780323857390.

[49] AN Reis, AR Pitombeira-Neto, and GA Rolim. Simulation of tank truck loading operations in a fuel distribution terminal. *Int. J. Simul. Model*, 16: 435–447, 2017.

[50] C. J. C. H Watkins. *Learning from Delayed Rewards*. King's College, Cambridge United Kingdom, 1989.

[51] Hado Hasselt. Double q-learning. In J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 23. Curran Associates, Inc., 2010.

[52] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.